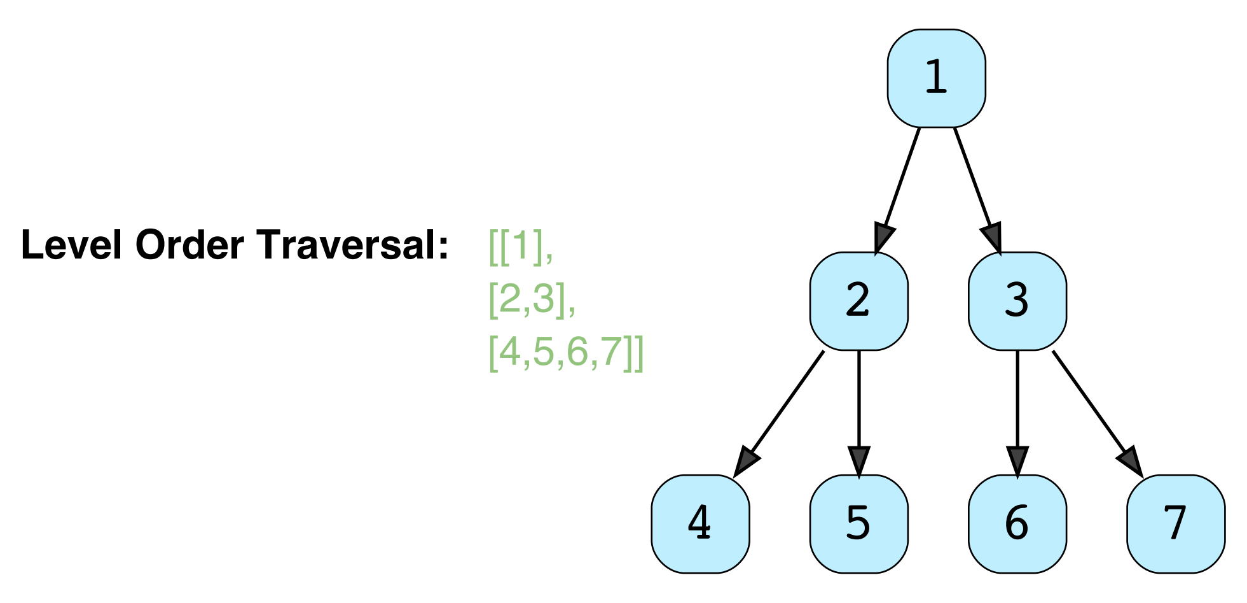
Solution: Binary Tree Level Order Traversal

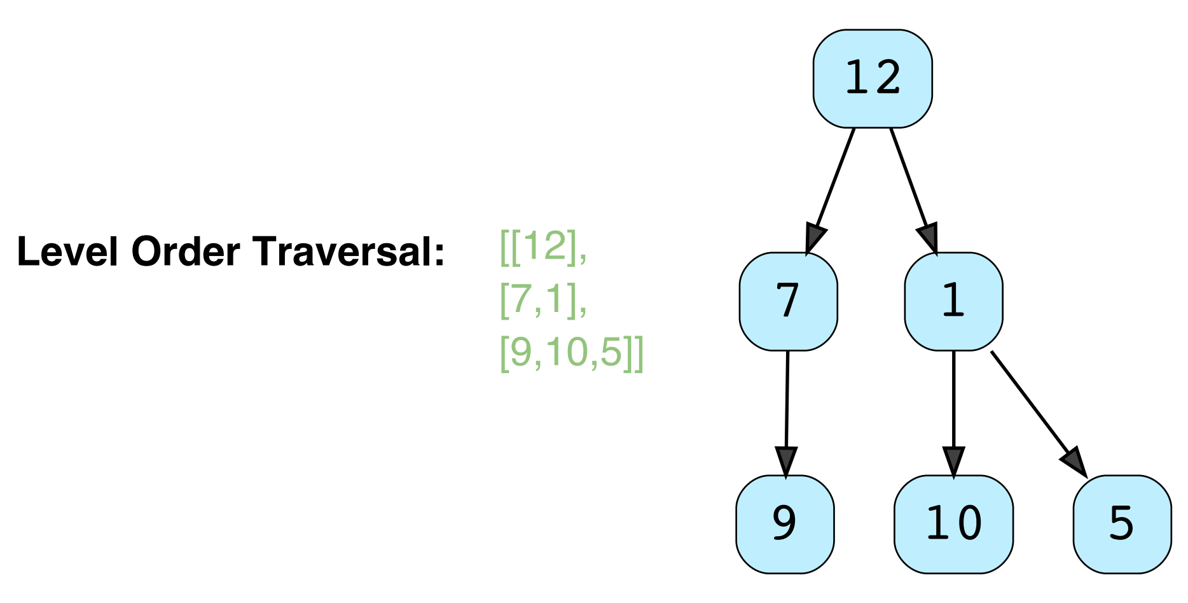
Problem Statement

Given a binary tree, populate an array to represent its level-by-level traversal. You should populate the values of all **nodes of each level from left to right** in separate sub-arrays.

Example 1:


Level Order Traversal:
[[1],
[2,3],
[4,5,6,7]]
1
2
3
4
5
6
7

Example 2:


Level Order Traversal:
[[12],
[7,1],
[9,10,5]]
12
7
1
9
10
5

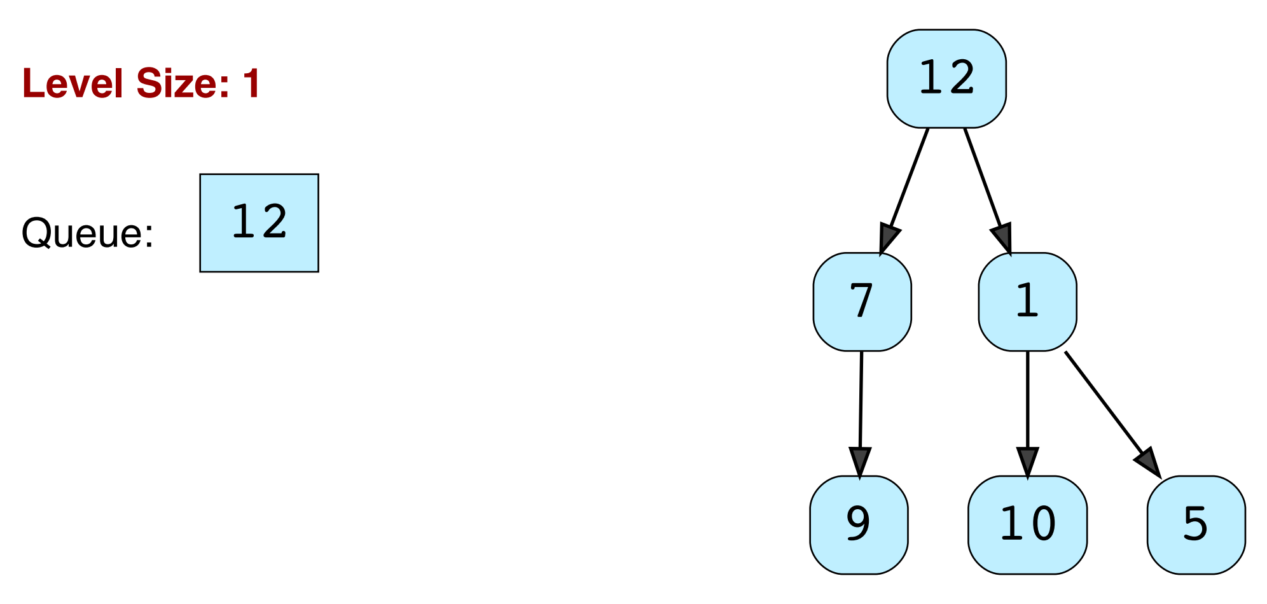
Solution

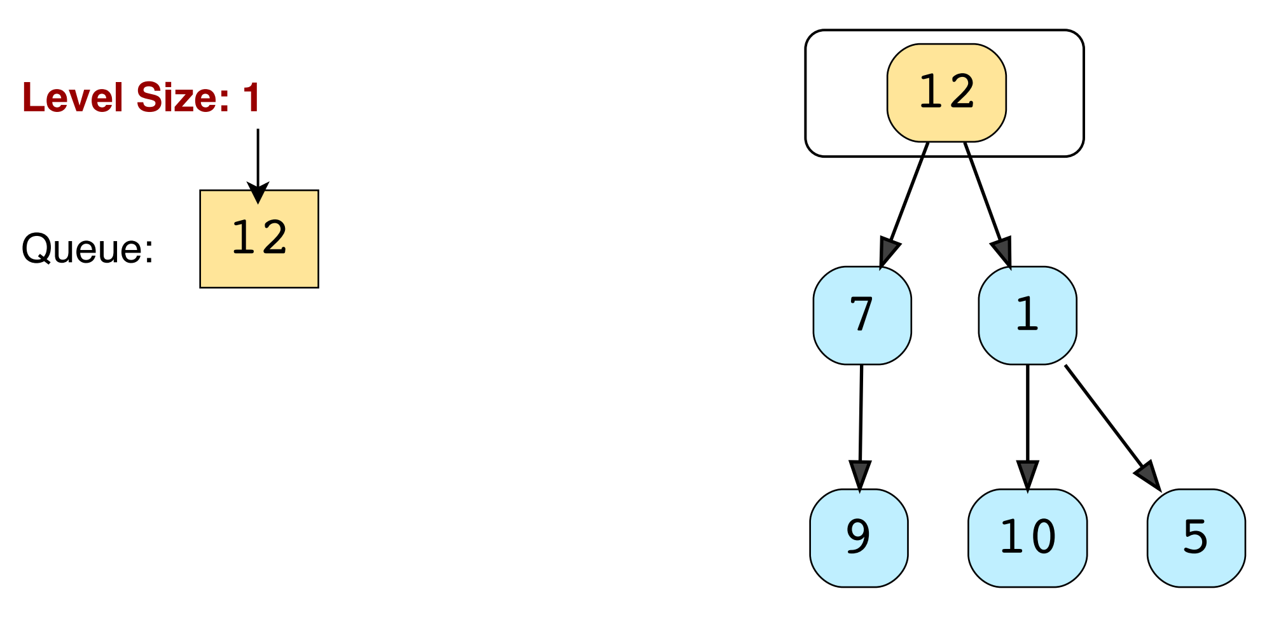
Since we need to traverse all nodes of each level before moving onto the next level, we can use the **Breadth First Search (BFS)** technique to solve this problem.

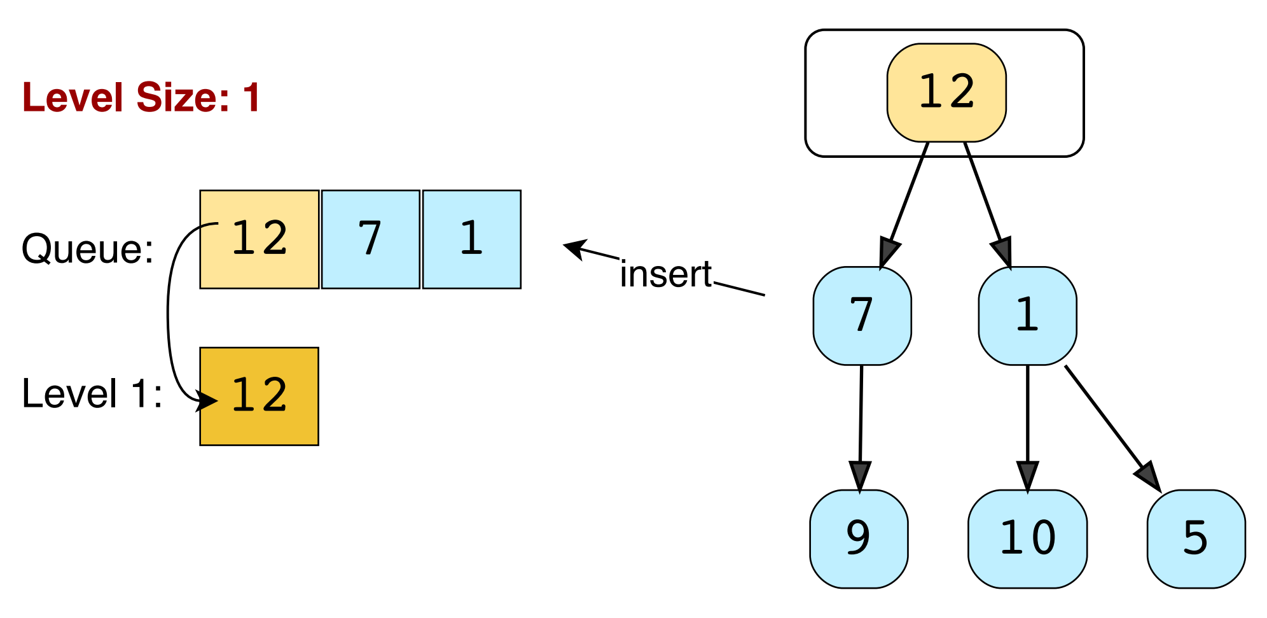
We can use a Queue to efficiently traverse in BFS fashion. Here are the steps of our algorithm:

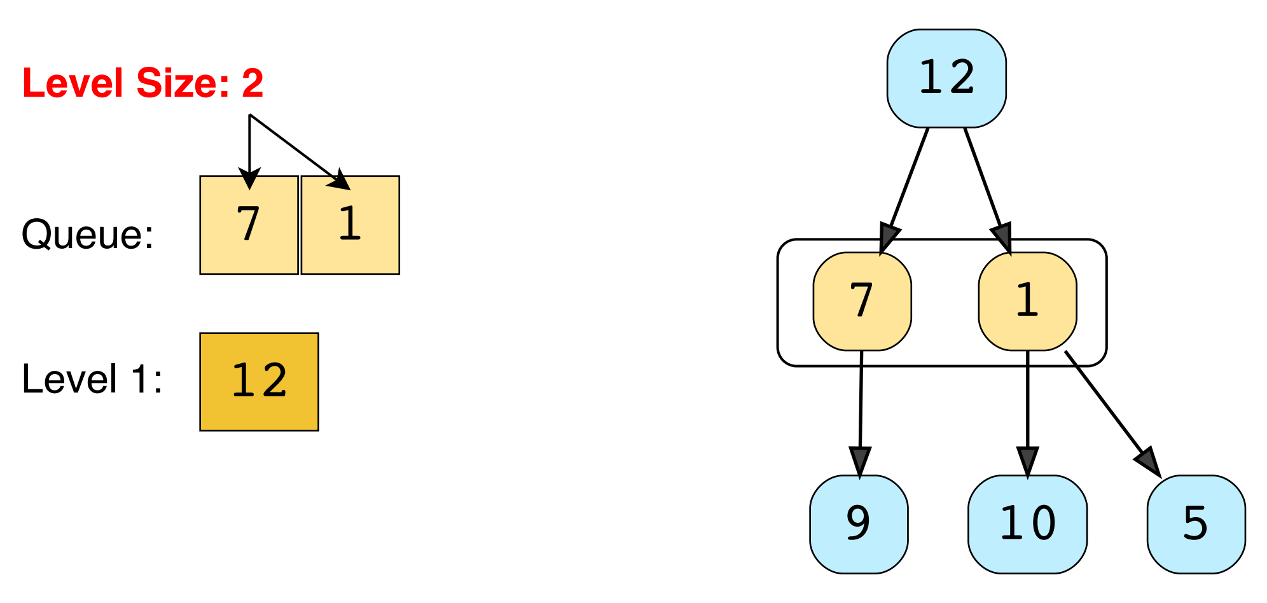
1. Start by pushing the `root` node to the queue.
2. Keep iterating until the queue is empty.
3. In each iteration, first count the elements in the queue (let's call it `levelSize`). We will have these many nodes in the current level.
4. Next, remove `levelSize` nodes from the queue and push their value in an array to represent the current level.
5. After removing each node from the queue, insert both of its children into the queue.
6. If the queue is not empty, repeat from step 3 for the next level.

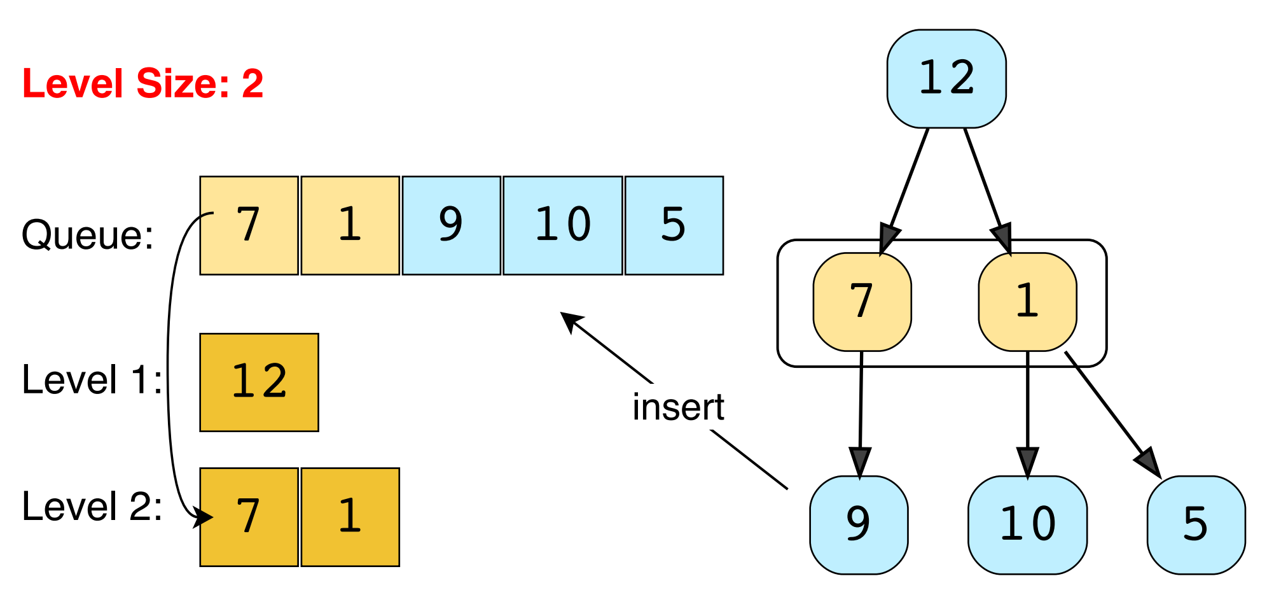
Let's take the example-2 mentioned above to visually represent our algorithm:

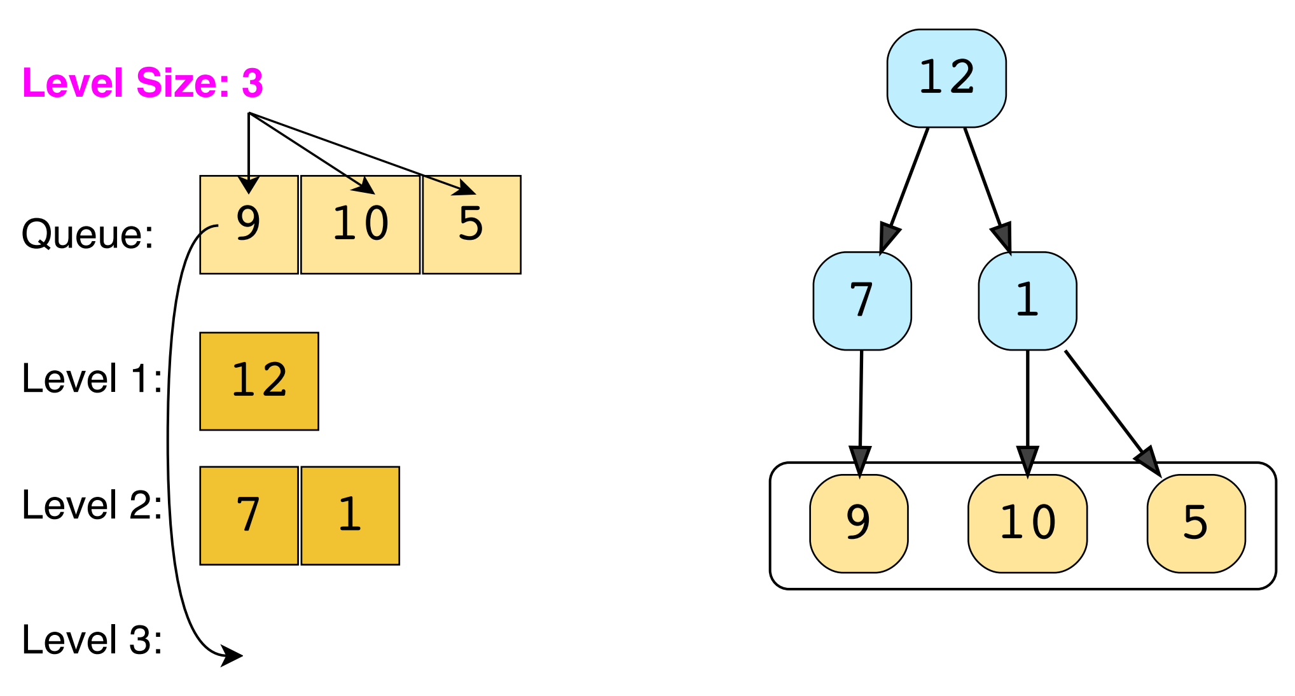

Level Size: 1
Queue:
12
12
7
1
9
10
5

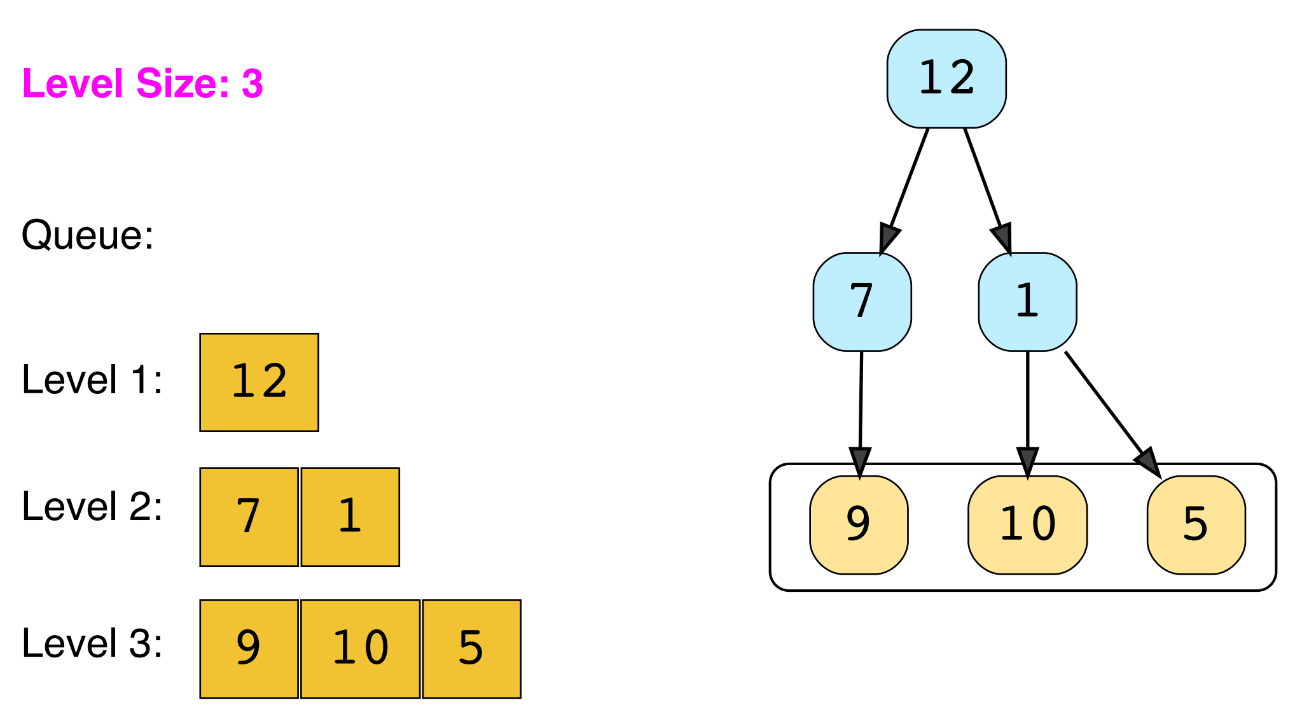

Level Size: 1
Queue:
12
12
7
1
9
10
5


Level Size: 1
Queue:
12
7
1
insert
Level 1:
12
12
7
1
9
10
5


Level Size: 2
Queue:
7
1
Level 1:
12
12
7
1
9
10
5

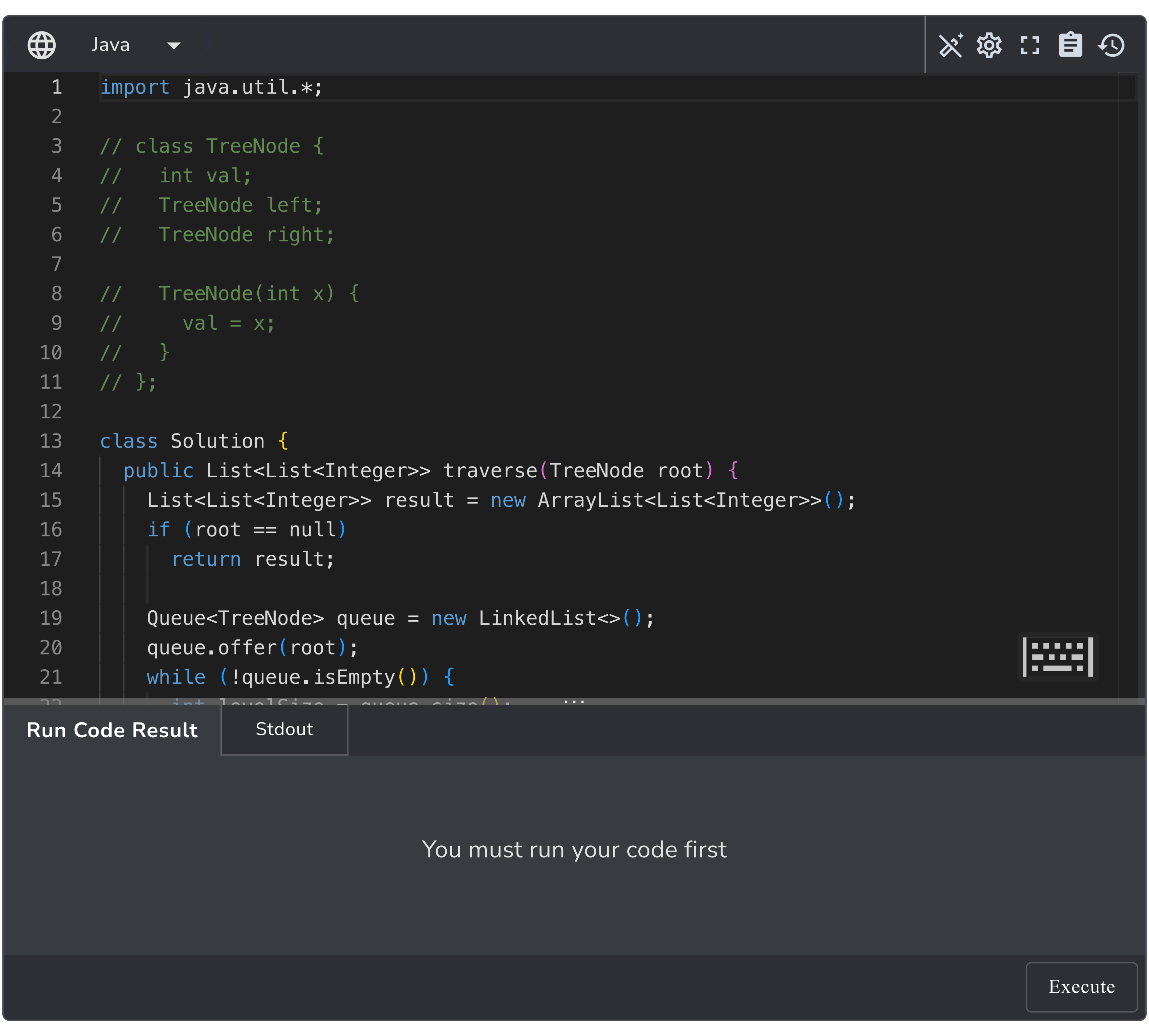

Level Size: 2
Queue:
7
1
9
10
5
Level 1:
12
insert
Level 2:
7
1
12
7
1
9
10
5


Level Size: 3
Queue:
9
10
5
Level 1:
12
Level 2:
7
1
Level 3:
12
7
1
9
10
5


Level Size: 3
Queue:
Level 1:
12
Level 2:
7
1
Level 3:
9
10
5
12
7
1
9
10
5

Code

Here is what our algorithm will look like:


Java

```java
import java.util.*;

// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;

//   TreeNode(int x) {
//     val = x;
//   }
// };

class Solution {
  public List<List<Integer>> traverse(TreeNode root) {
    List<List<Integer>> result = new ArrayList<List<Integer>>();
    if (root == null)
      return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
```

Run Code Result
Stdout

You must run your code first

Execute

Time Complexity

The time complexity of the above algorithm is O(N), where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space Complexity

The space complexity of the above algorithm will be O(N) as we need to return a list containing the level order traversal. We will also need O(N) space for the queue. Since we can have a maximum of N/2 nodes at any level (this could happen only at the lowest level), therefore we will need O(N) space to store them in the queue.

← Previous
Binary Tree Level Order Traversal
(Easy)

Next →
Reverse Level Order Traversal
(Easy)

☐ Mark as Completed